

计算机图形学

实验一： 三维渲染系统安装与使用

实验内容

- 在本地安装并部署授课组开发的服务与教学的渲染演示编程框架程序;
- 在本地安装并部署OpenGL环境;
- 熟悉渲染框架程序的基本使用;
- 实现自定义渲染算法。

环境配置

- 操作系统: Windows 10
- 编译器: MSVC(Visual Studio 2019以上)
- Opengl 3.3 以上
- CMake 3.18以上

编译说明

- 克隆代码到仓库：
 - git clone <https://gitee.com/nankaigraphics/nrenderer-comment.git>
 - 或浏览<https://gitee.com/nankaigraphics/nrenderer-comment.git>下载ZIP文件并解压

VisJiao add comments 45ef328 9个月前 67 次提交

└ .vs	material refactor	5年前
└ .vscode	env_map	4年前
└ code	add comments	9个月前
└ doc	env_map	4年前
└ resource	env_map	4年前
└ .gitignore	.gitignore	4年前
└ Readme.md	..	4年前

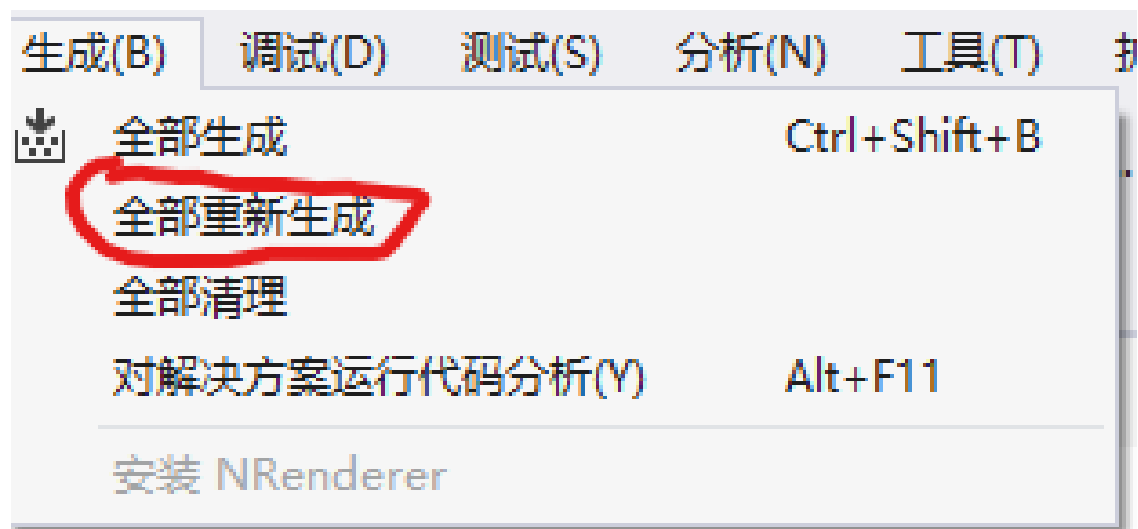
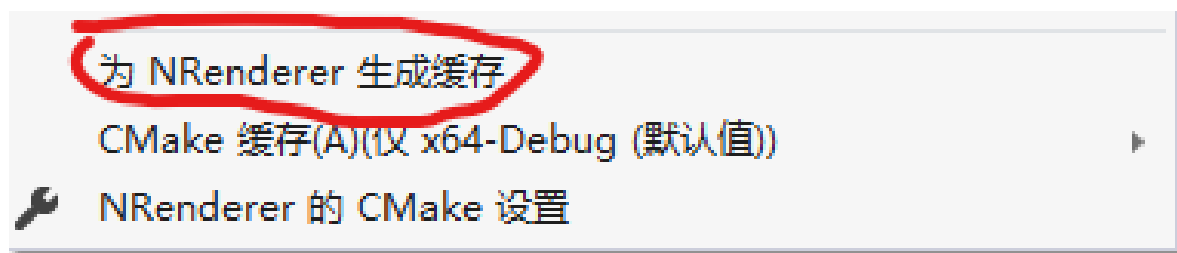
README

- 服务于本科教学的三...
- 环境配置
- 编译说明

服务于本科教学的三维渲染系统

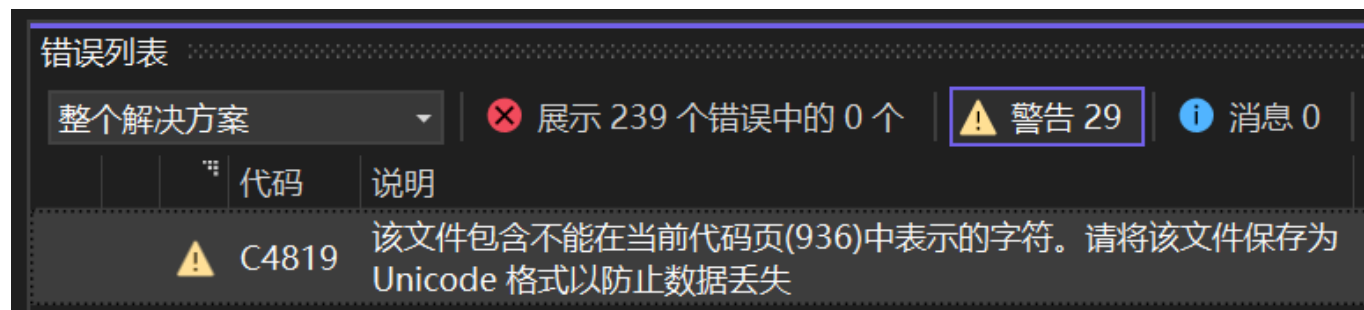
编译说明

- 打开./code所在文件夹->右键->使用Visual Studio打开
- 项目->生成CMake缓存
- 生成->全部重新生成



编译说明

- 如果遇到下图所示的警告



- 需要将出现警告的文件保存为以下编码，可以消除由于格式问题引起的错误

编码(E):

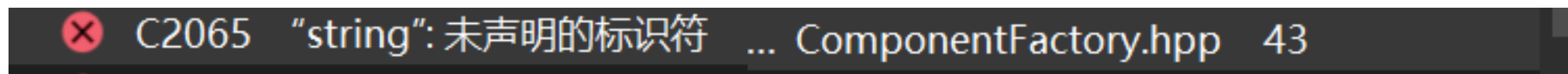
简体中文(GB2312) - 代码页 936

行尾(L):

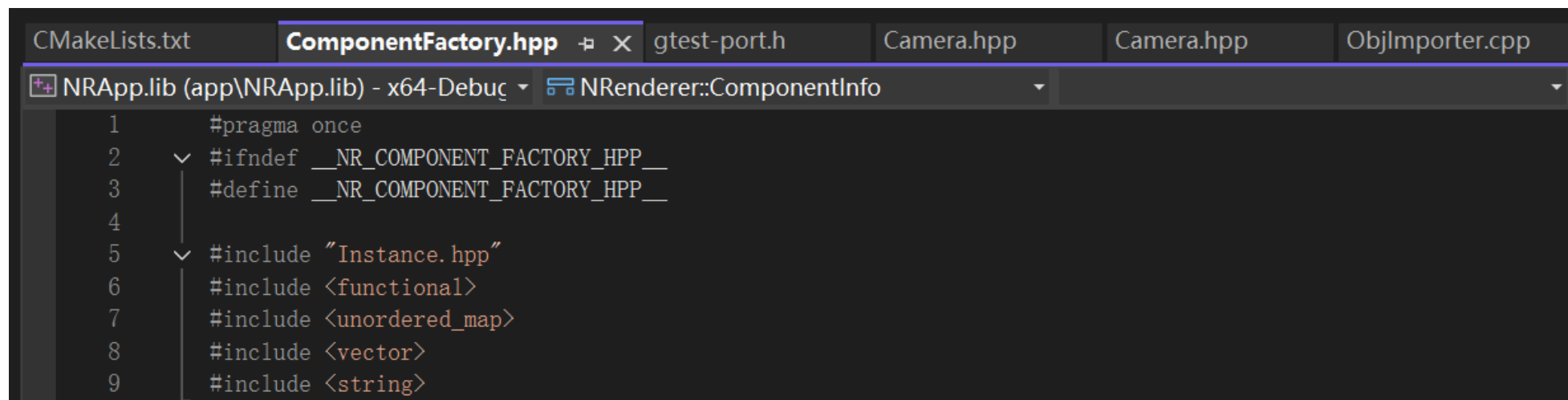
当前设置

编译说明

- 如果遇到下图所示的错误



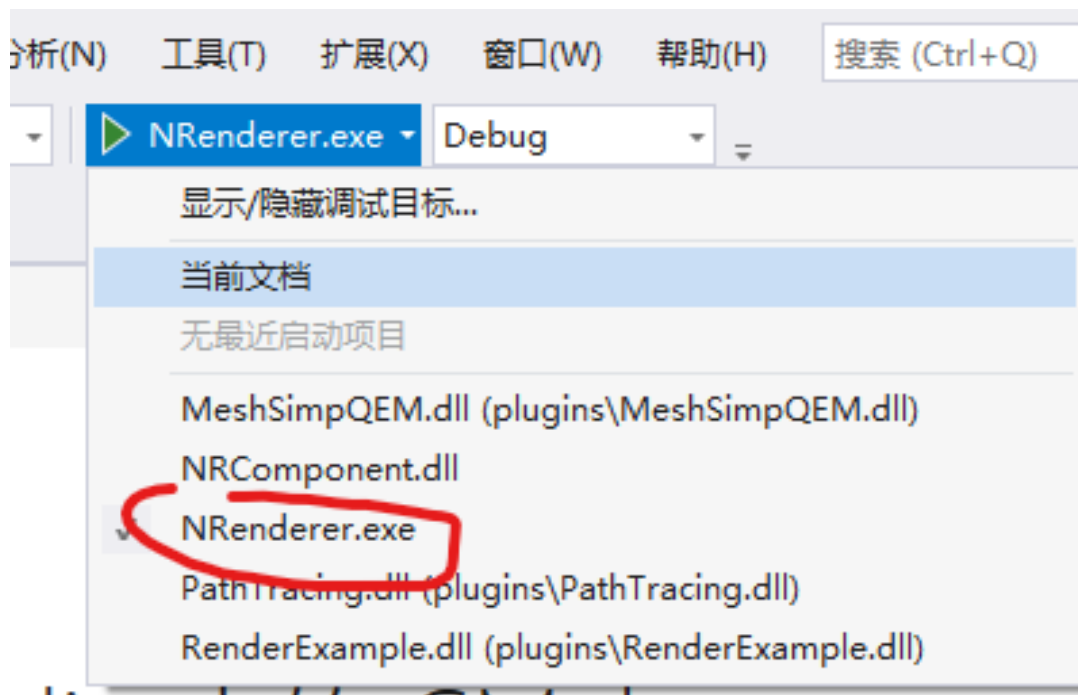
- 需要在ComponentFactory.hpp文件中加入#include <string>



注：实验室机房的代码在D盘下的CG/nrenderer-master文件夹中，已经配置好了全部所需环境，可以直接使用

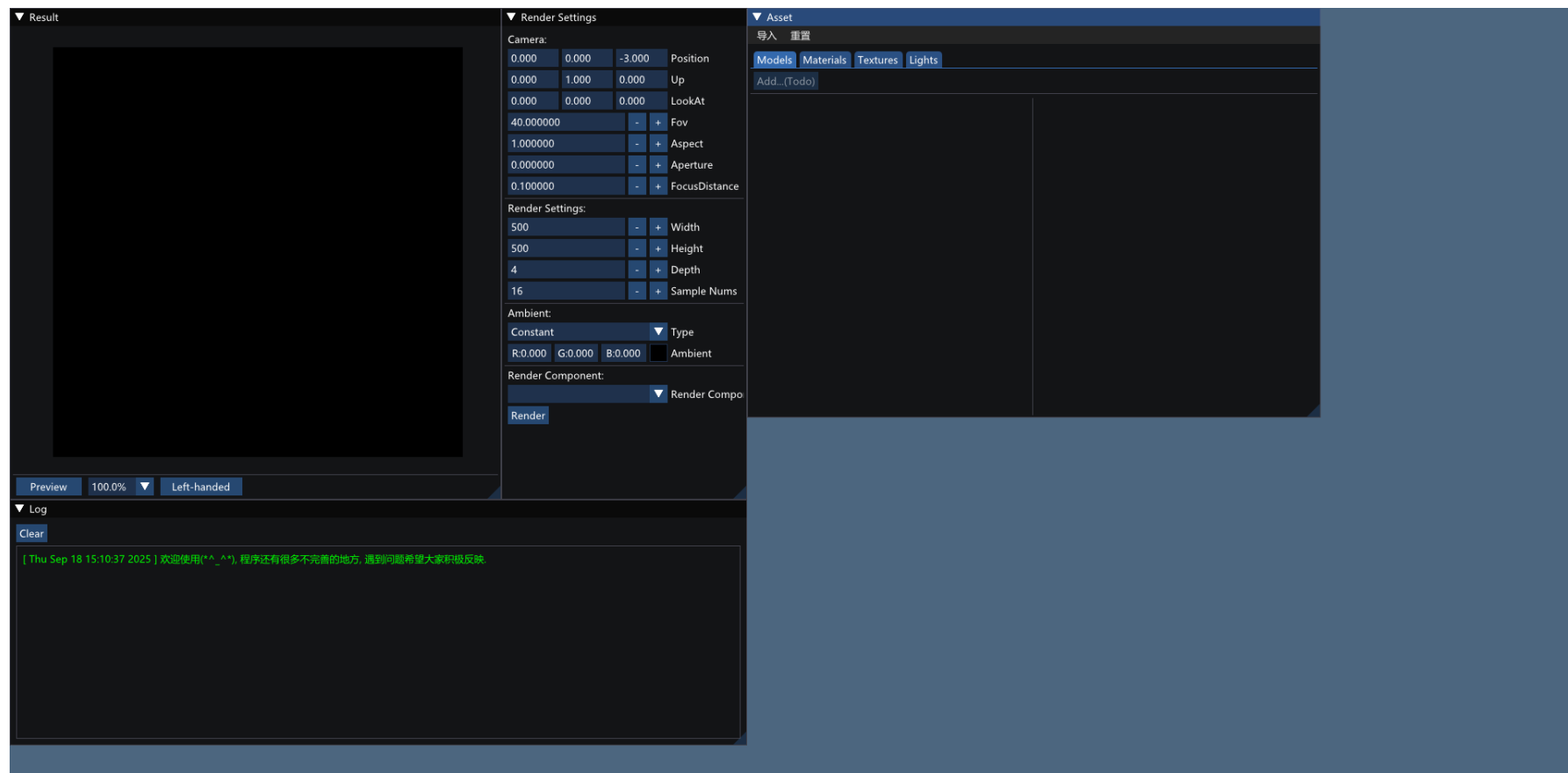
编译说明

- 选择启动项



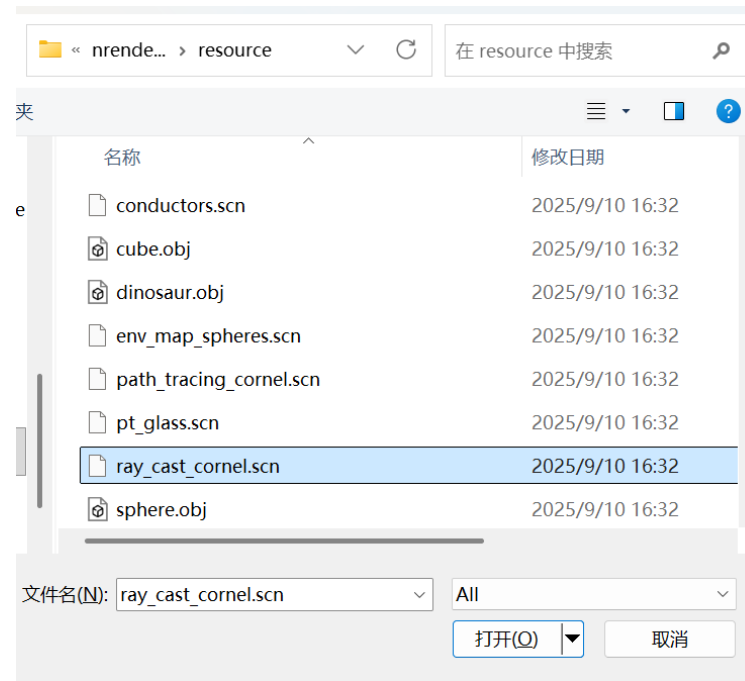
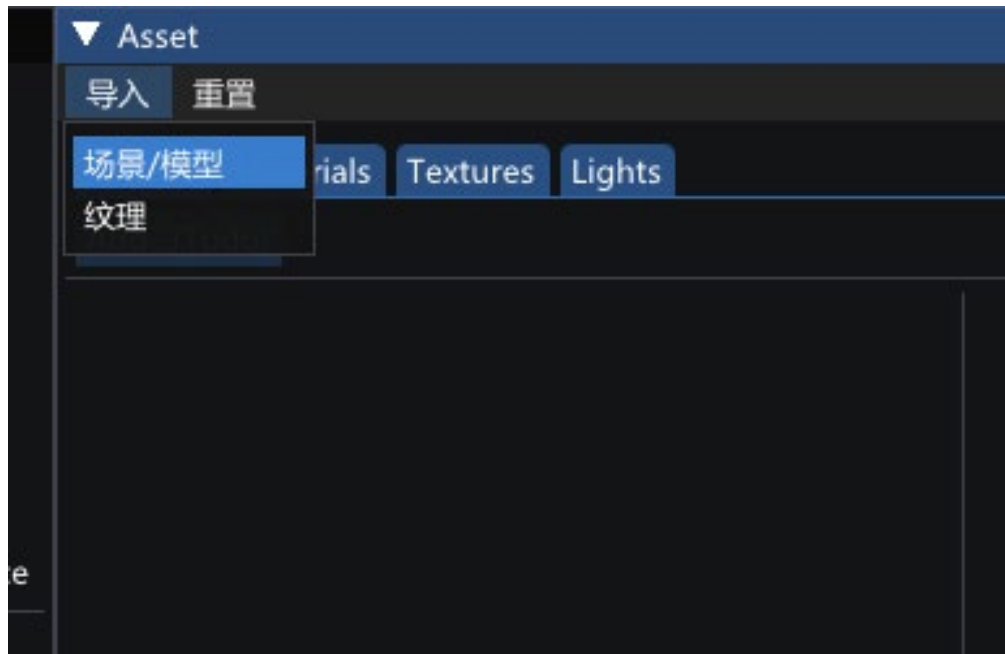
- 注: 不喜欢用CMake的同学可以通过CMake生成vs工程使用

编译说明



- 成功启动的界面

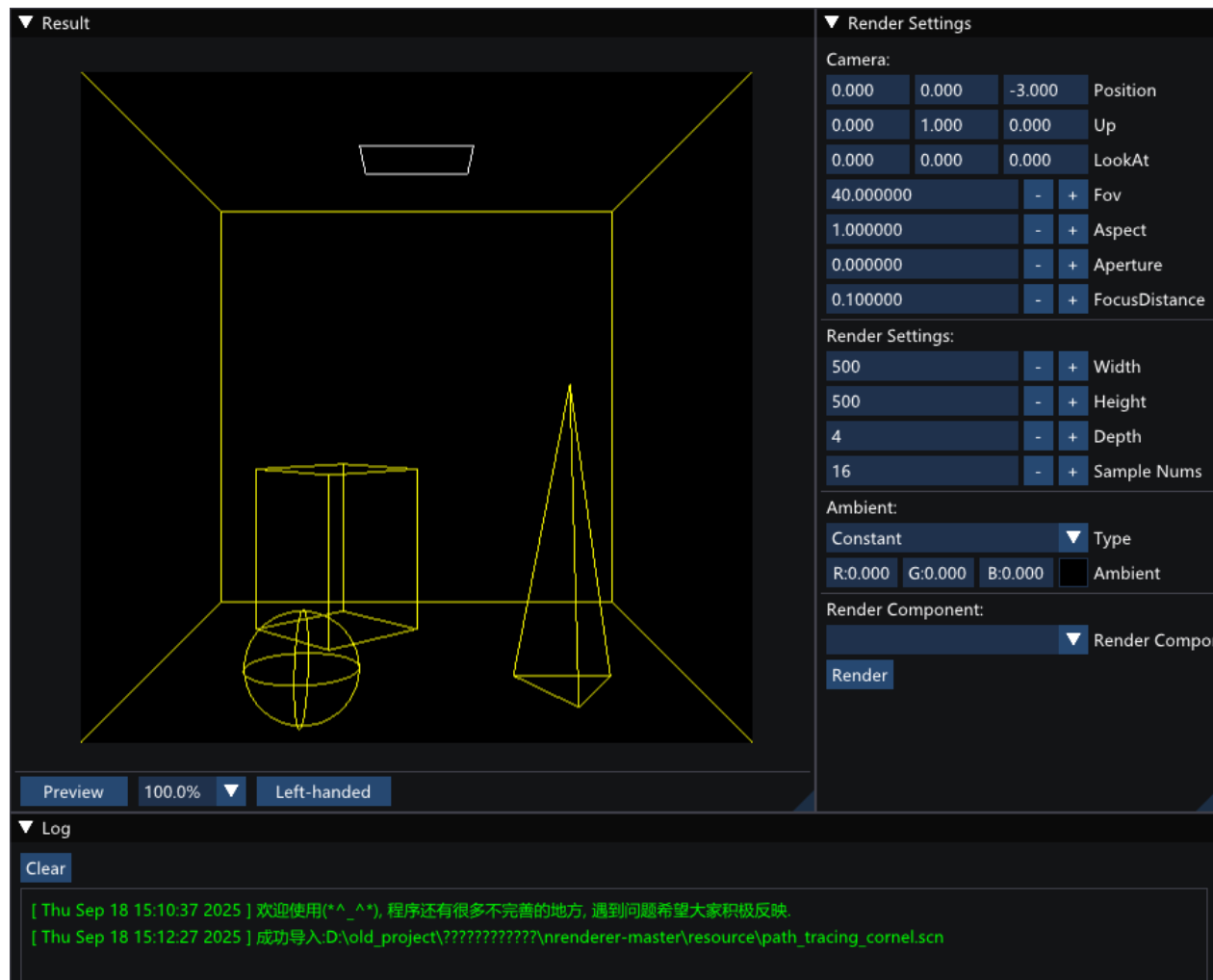
使用方法-导入场景文件



- 点击Asset部分中的“导入”选项，选择下拉菜单中的“场景/模型”进行导入；
导入的文件位于nrenderer-master/resource文件夹内
- 作为示例，我们导入ray_cast_cornel.scn场景文件

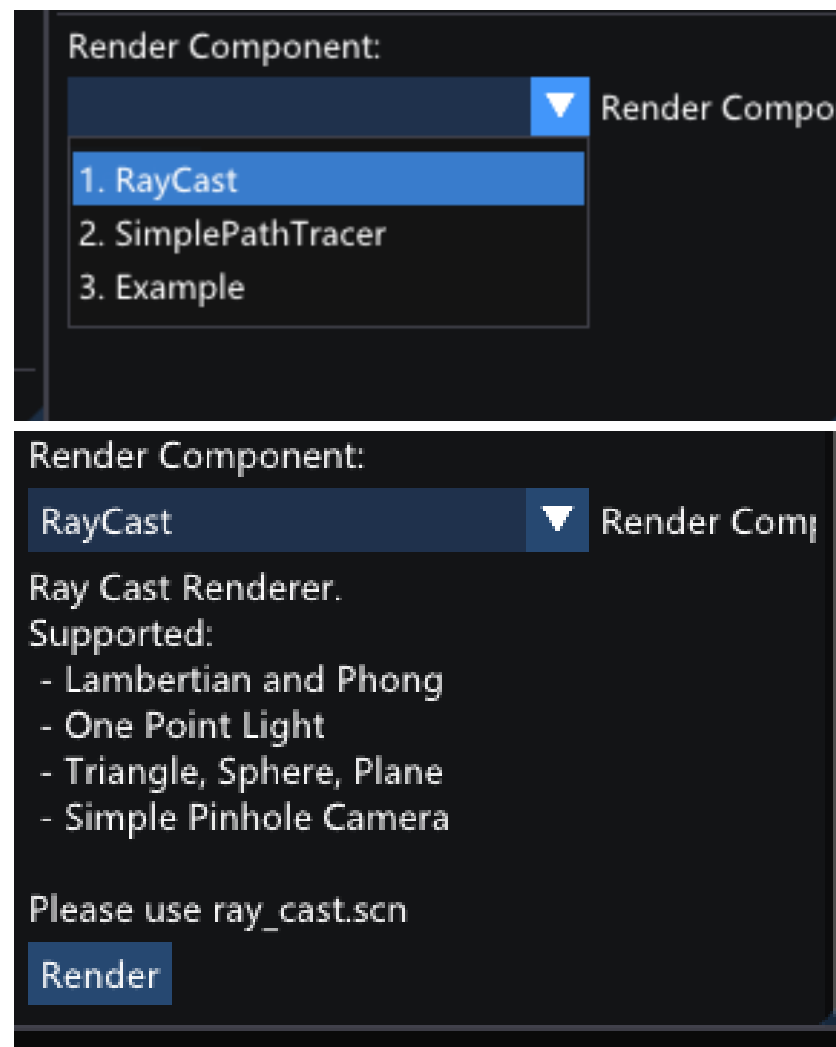
使用方法-快速预览

- 成功导入场景文件后，Result部分用线条简单显示出预览效果



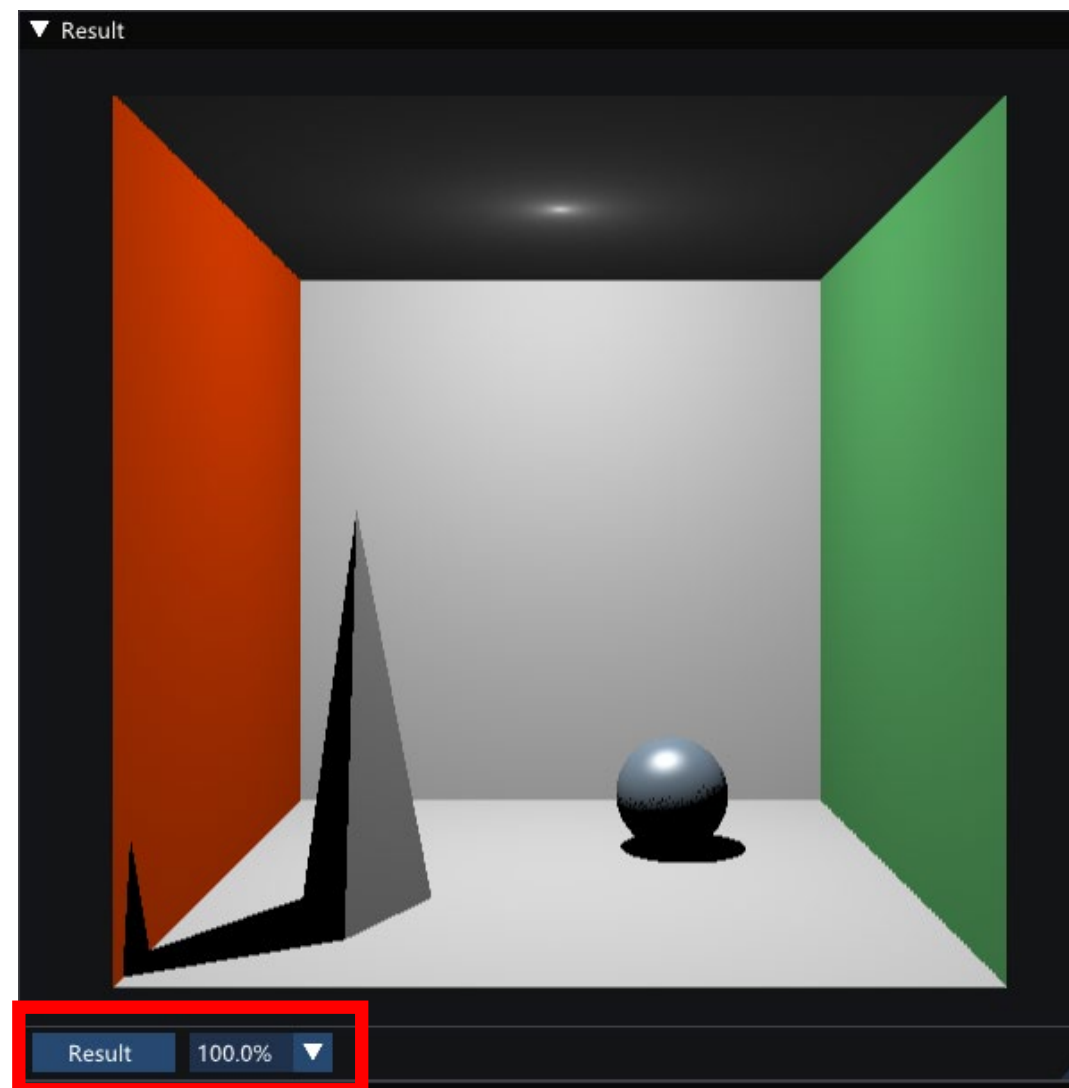
使用方法-选择渲染方法并渲染

- 在Render Settings部分的Render Component选择RayCast作为渲染方法
- 选好之后点击下方“Render”键即可得到默认参数下使用RayCast作为渲染方法的渲染结果



使用方法-切换渲染结果/快速预览

- 此时得到右侧的渲染结果
- 点击左下角的Result可以切换渲染结果和快速预览的效果
- 通过下拉菜单更改左下角的数字（图中为100%）可以改变显示图片的大小



使用方法-修改场景参数

- Camera:
 - Position: 相机的位置坐标, 即相机在三维空间中的位置点
 - Up: 相机的“上方向”向量, 用来确定相机的朝向和画面不歪斜
 - LookAt: 相机注视的目标点坐标, 相机的“视线”就是从Position指向LookAt

Render Settings				
Camera:				
0.000	0.000	-3.000	Position	
0.000	1.000	0.000	Up	
0.000	0.000	0.000	LookAt	
40.000000	-	+	Fov	
1.000000	-	+	Aspect	
0.000000	-	+	Aperture	
0.100000	-	+	FocusDistance	

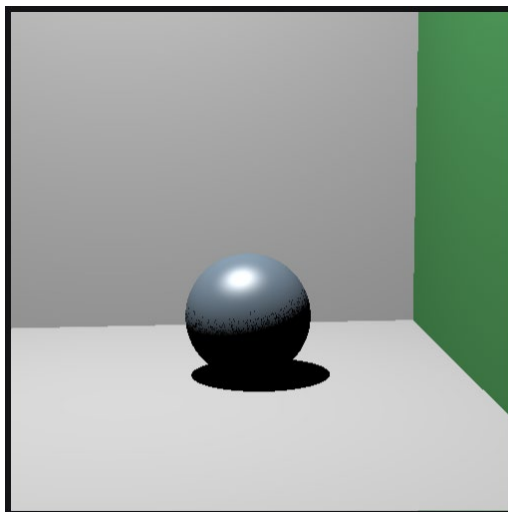
使用方法-修改场景参数

- Camera:
 - Fov: 相机能看到的视角范围
 - Aspect: 图像的宽度与高度的比例, 决定渲染画面的纵横比
 - Aperture: 光圈大小, 对应景深效果
 - FocusDistance: 相机对焦的距离

Render Settings				
Camera:				
0.000	0.000	-3.000	Position	
0.000	1.000	0.000	Up	
0.000	0.000	0.000	LookAt	
40.000000	-	+	Fov	
1.000000	-	+	Aspect	
0.000000	-	+	Aperture	
0.100000	-	+	FocusDistance	

使用方法-修改场景参数

- 通过调整相机的参数，可以改变最终获得的结果
- 将相机参数改为右侧的参数，可以把结果从整个场景变为仅聚焦于球体的效果



Render Settings				
Camera:				
0.300	0.500	-3.000	Position	
0.000	1.000	0.000	Up	
0.000	0.000	0.000	LookAt	
20.000000	-	+	Fov	
1.000000	-	+	Aspect	
0.000000	-	+	Aperture	
0.100000	-	+	FocusDistanc	

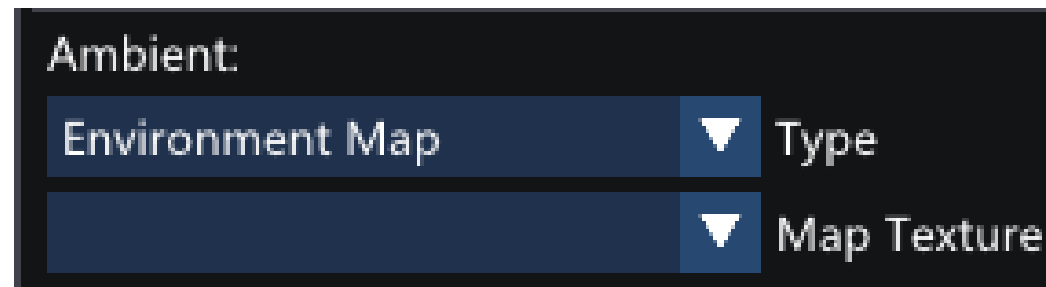
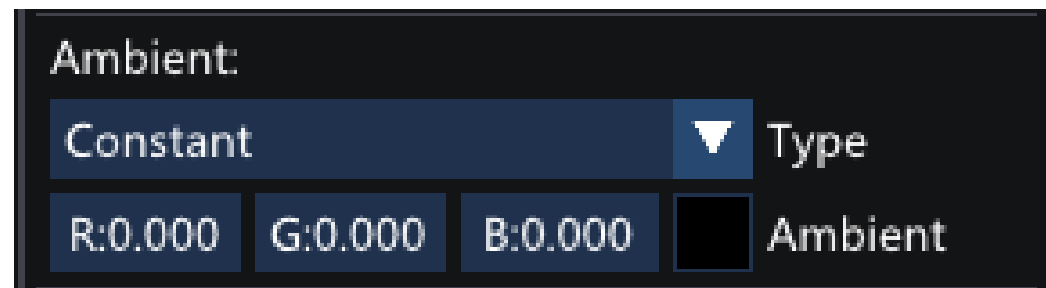
使用方法-修改场景参数

- Render Settings:
 - Width: 渲染结果的宽度
 - Height: 渲染结果的高度
 - Depth: Simple Path Tracer渲染方法中光线反射/折射的最大次数
 - Sample Nums: Simple Path Tracer渲染方法中Monte Carlo方法的采样数量

Render Settings:			
500	-	+	Width
500	-	+	Height
4	-	+	Depth
16	-	+	Sample Num

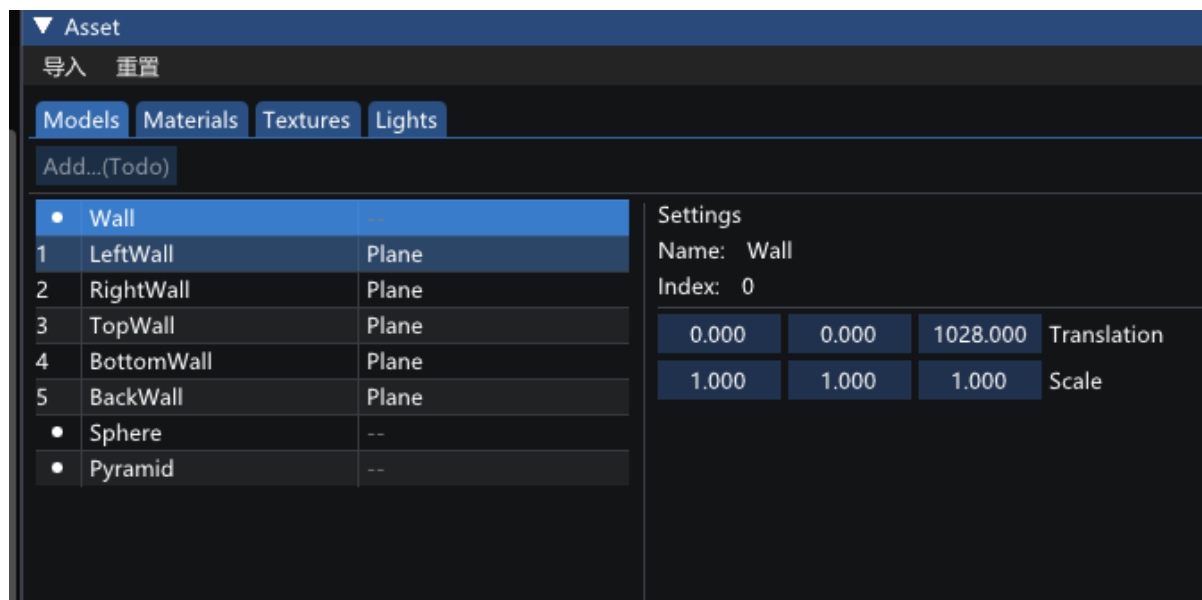
使用方法-修改场景参数

- Ambient:
 - Type: 环境类型
 - 固定颜色的环境Constant
 - 指定环境颜色RGB
 - 环境光Environment Map
 - 指定环境贴图



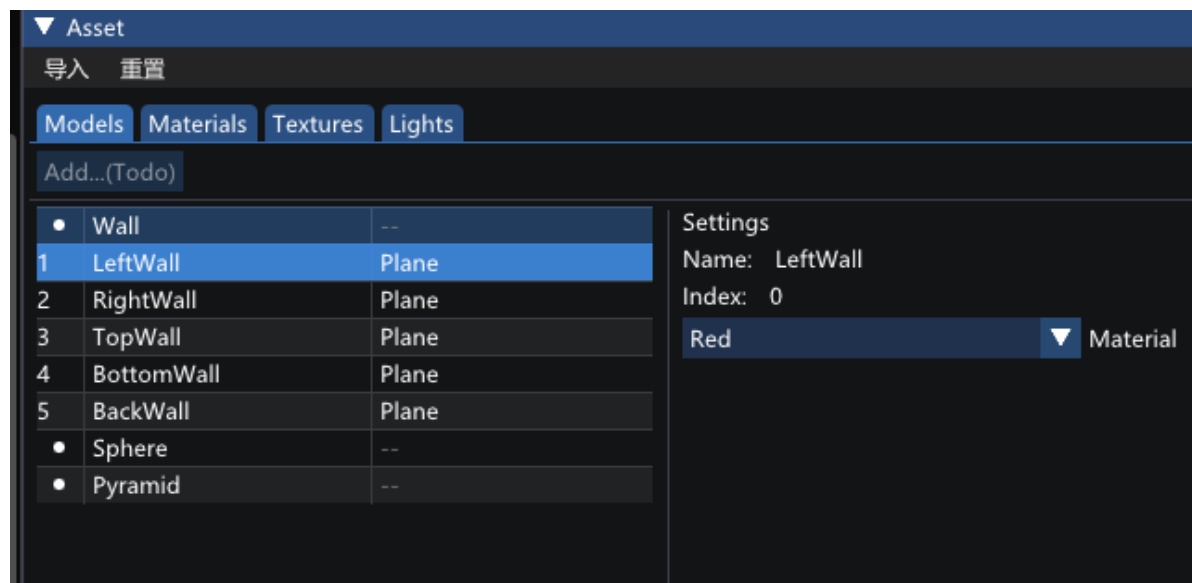
使用方法-修改场景参数

- Models:
 - 以Wall这个组合为例：
 - Wall组合包含各个位置的5面墙
 - Translation: 指定整个组合的位置
 - 修改数据的方式为点击之后鼠标左右滑动



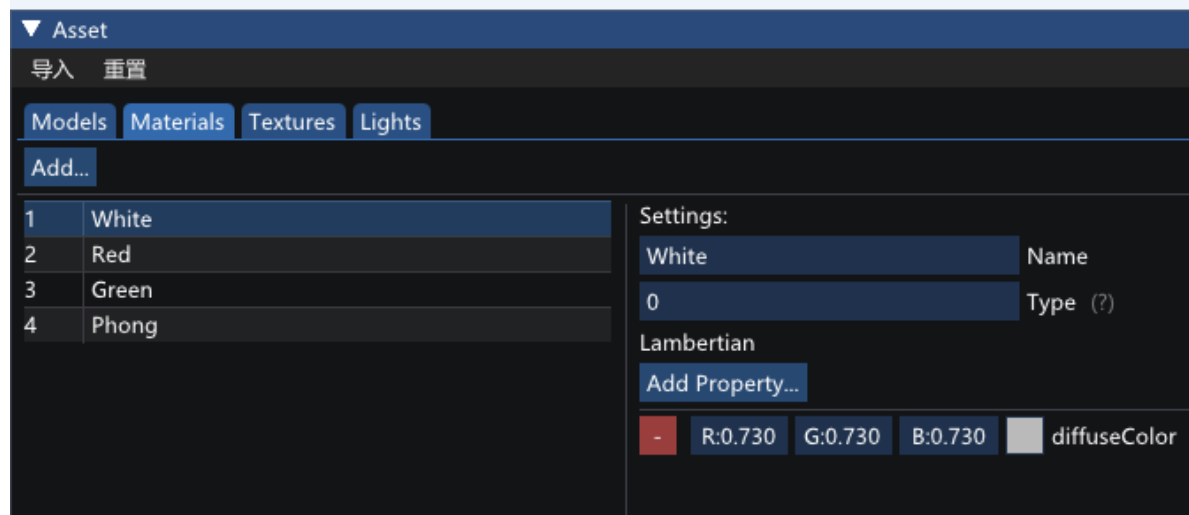
使用方法-修改场景参数

- Models:
 - Name: 模型名称
 - Index: 模型编号
 - Material: 指定材质类型



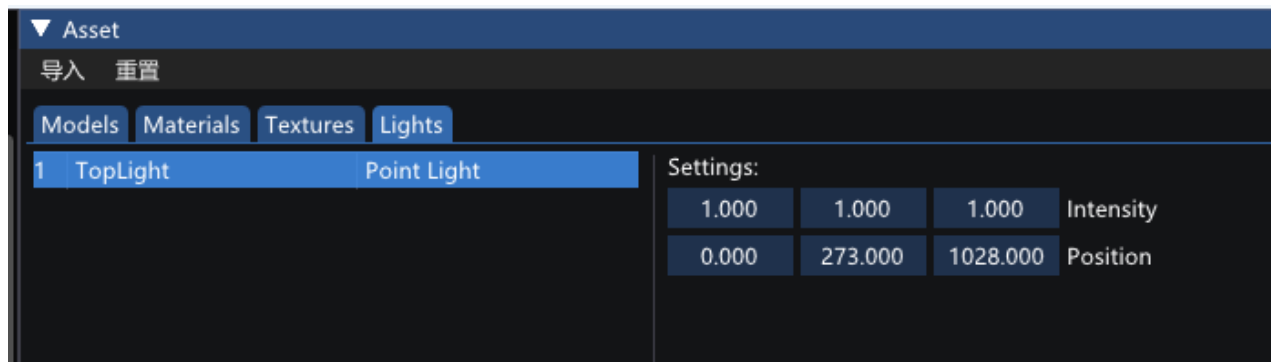
使用方法-修改场景参数

- Materials:
 - Name: 材质名称
 - Type: 材质类型
 - 下方文字 (Lambertian)
为当前类型id所对应的具体类型
 - Add Property: 为当前材质
添加物质特性



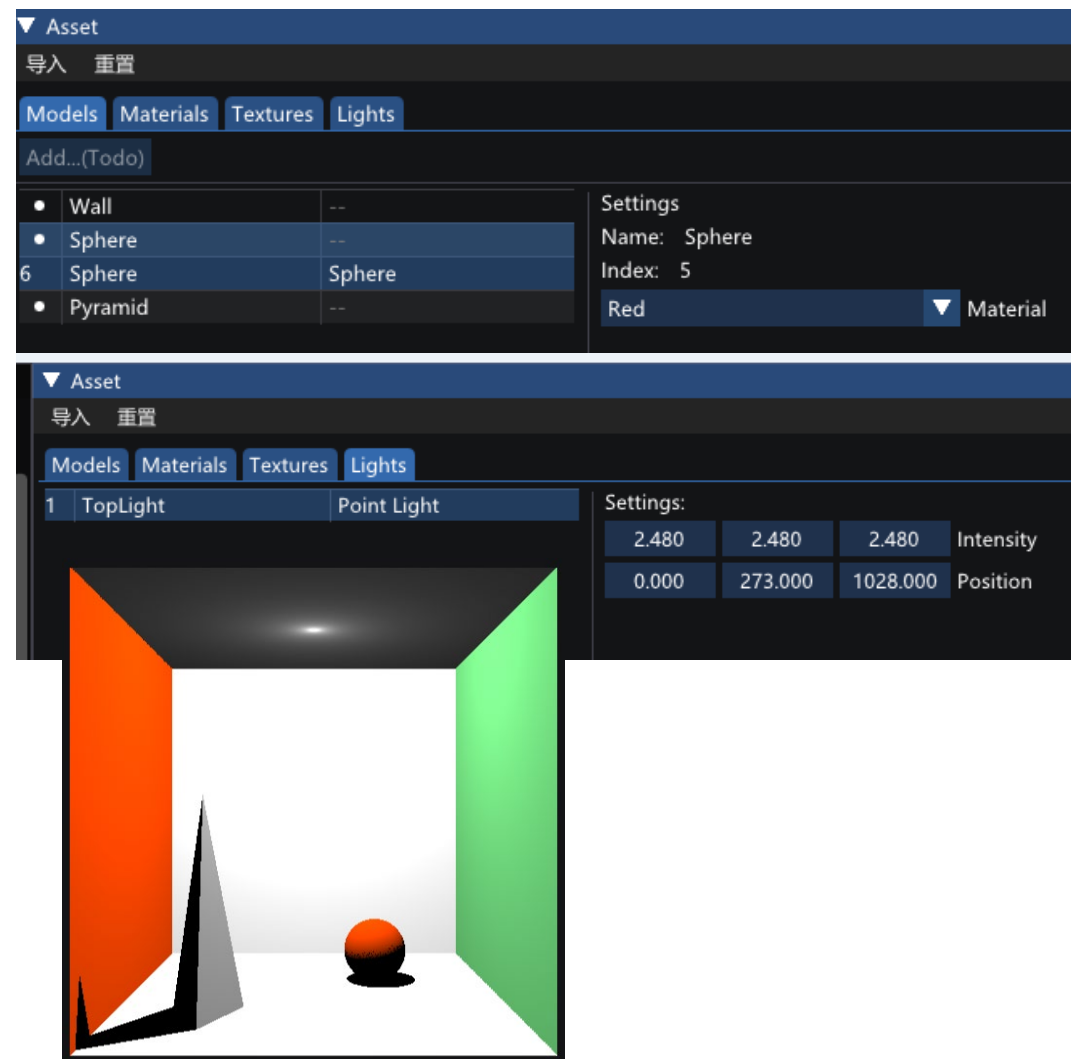
使用方法-修改场景参数

- Lights:
 - Intensity: 光照强度
 - Position: 光源位置



使用方法-修改场景参数

- 修改模型材质和场景参数，可以达到不同的效果
- 例如达到右图效果，参数调整为：
 - 场景： ray_cast_cornel.scn
 - 对象Name： Shpere
 - 材质Material： Red
 - 光源TopLight：
 - 光照强度Intensity： 2.48, 2.48, 2.48

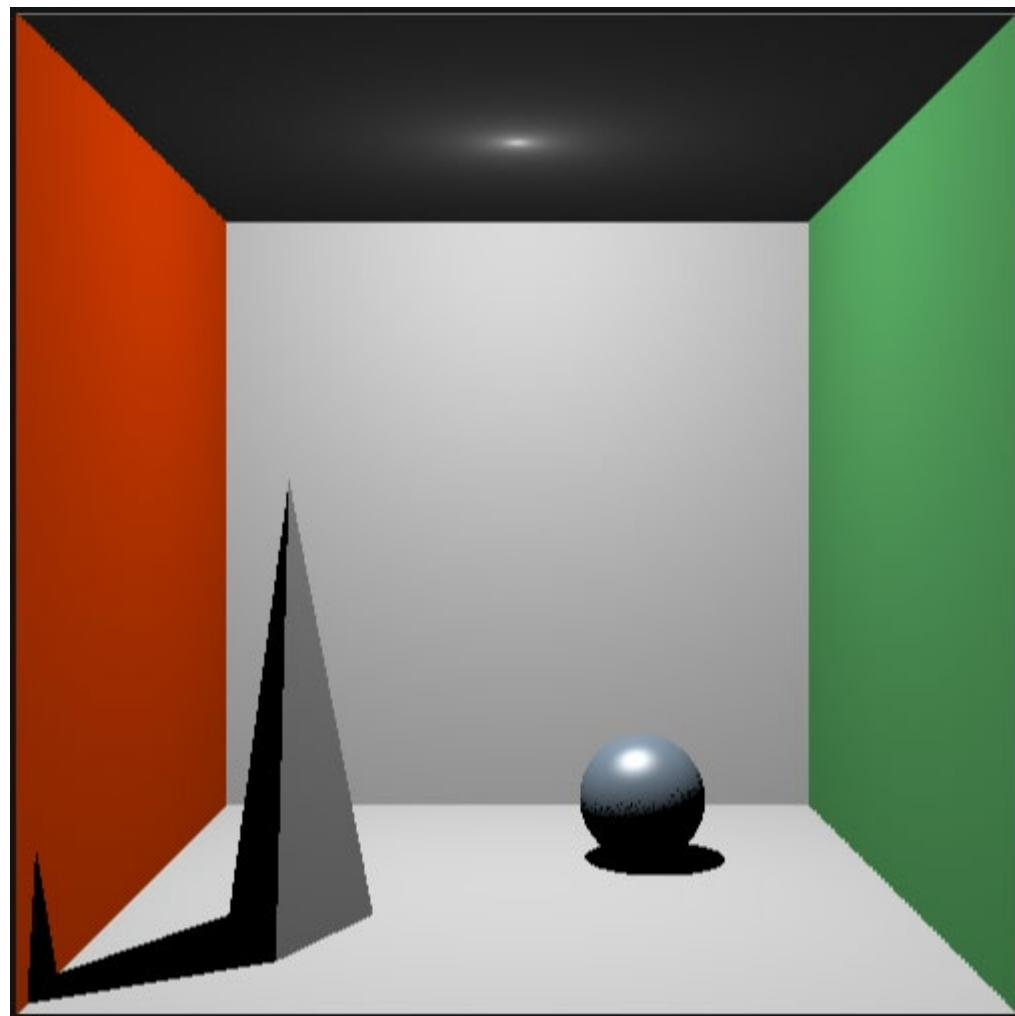


样例算法-Ray Cast

- 向场景中投射光线, 计算直接光照. 计算方法为Phong, 忽略环境光:

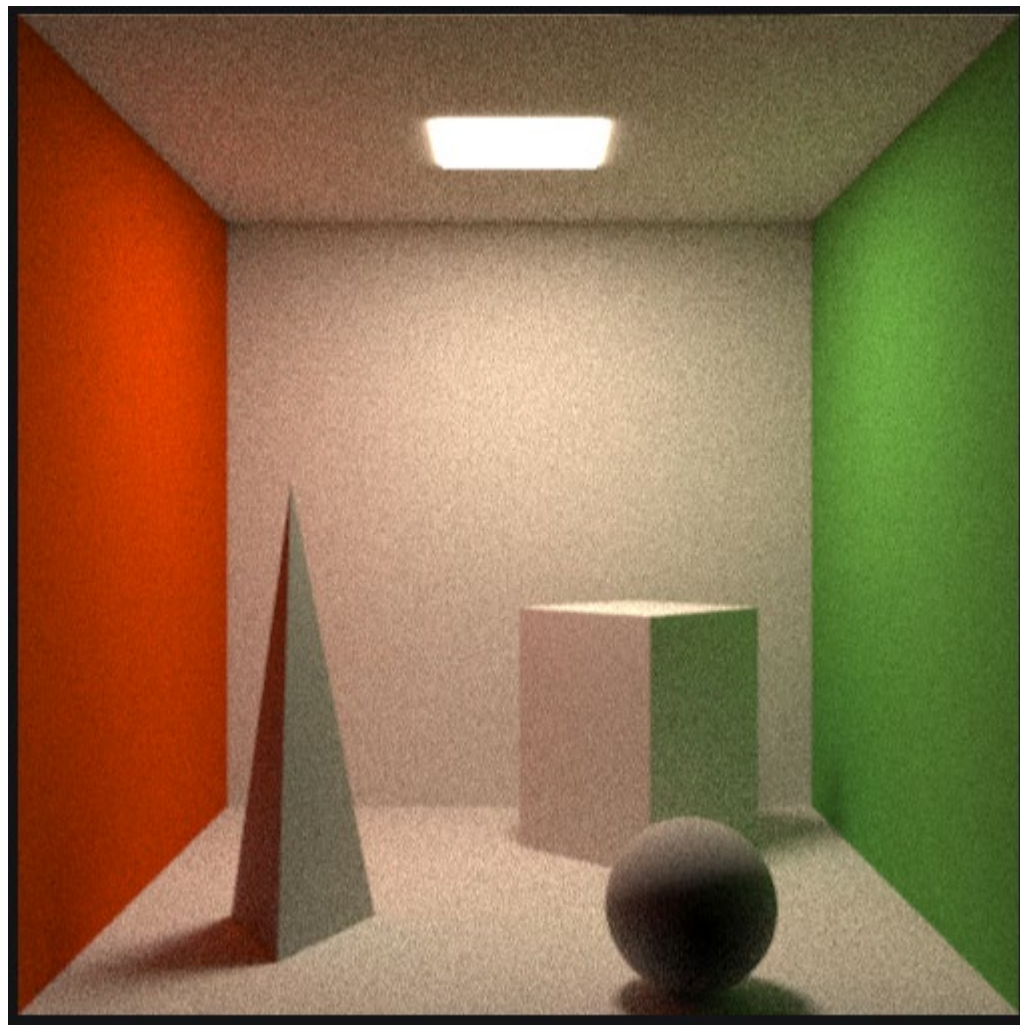
$$I = I_d k_d (L \cdot N) + I_s k_s (R \cdot V)^n$$

- 场景文件: ray_cast_cornel.scn
- 导入ray_cast_cornel.scn文件, 使用RayCast作为渲染方法, 即可得到右侧的效果

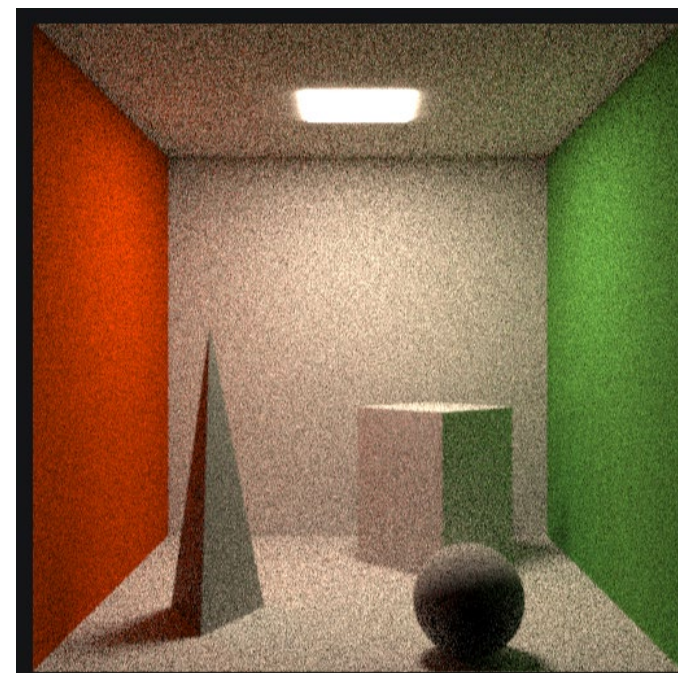
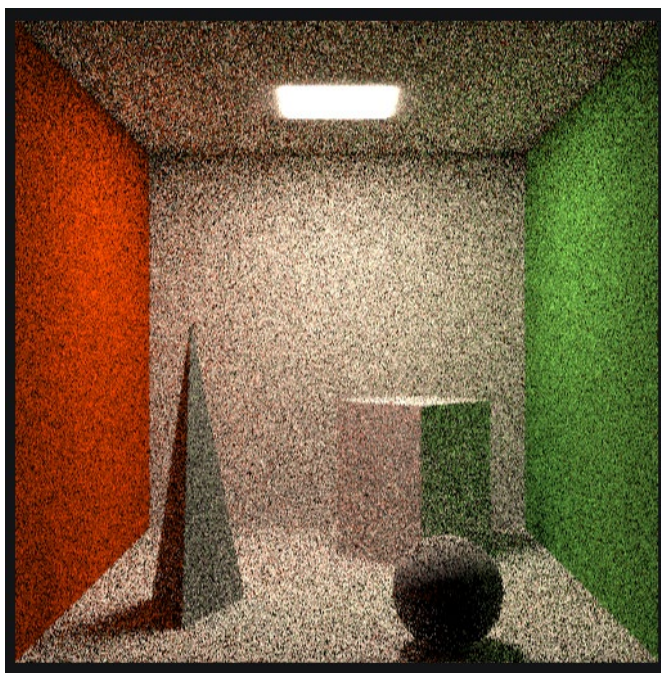
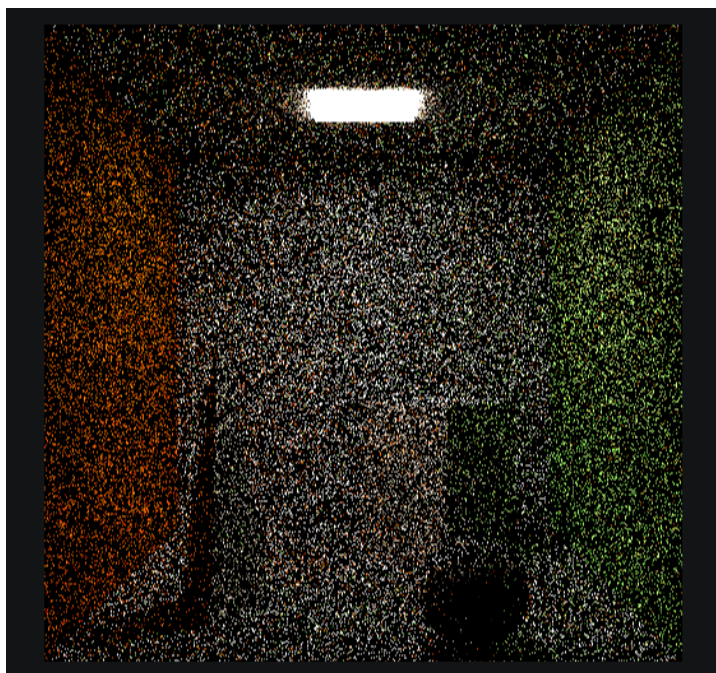


样例算法-Simple Path Tracer

- 使用Monte Carlo方法计算光照, 不支持网格, 仅支持漫反射材质
- 场景文件:
path_tracing_cornel.scn
- 导入path_tracing_cornel.scn文件, 使用SimplePathTracer作为渲染方法, 即可得到右侧的效果



样例算法-Simple Path Tracer

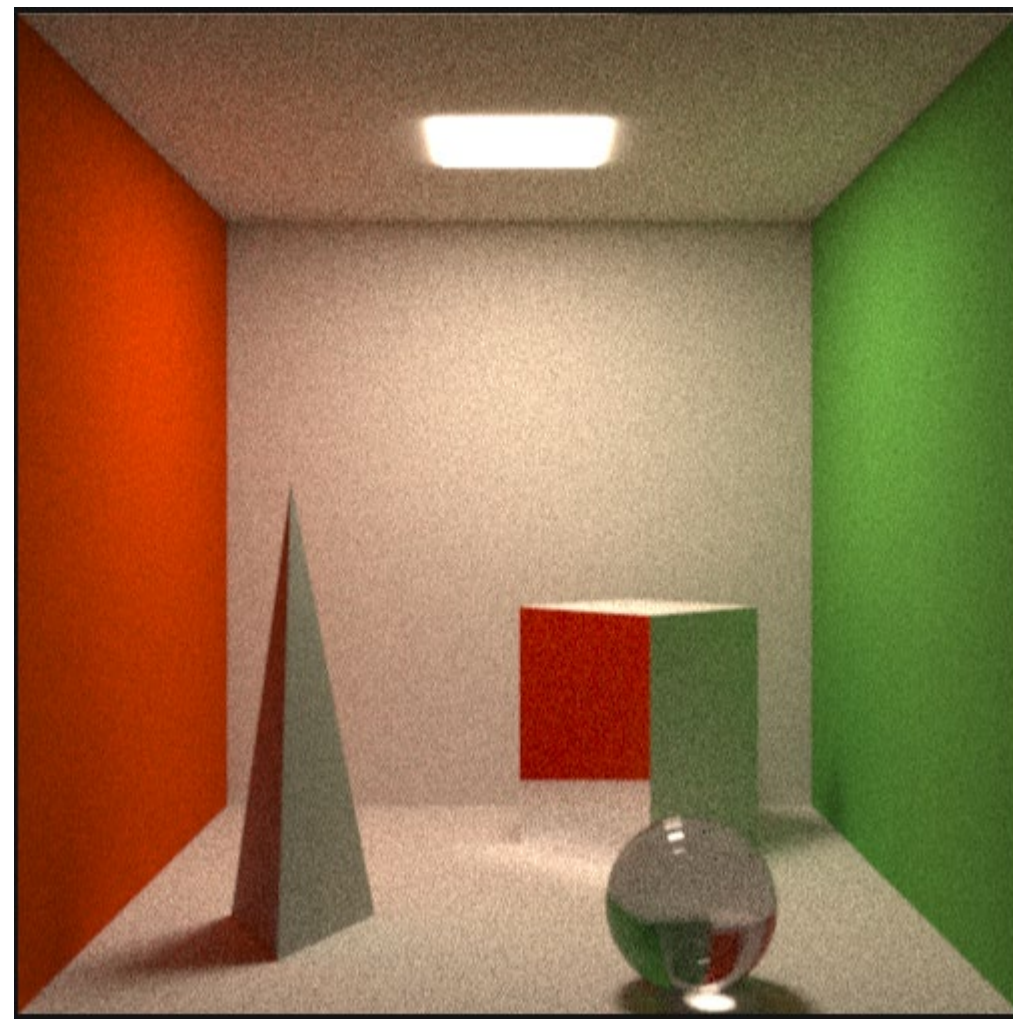


- 调整Sample Num数值，可以得到不同效果的渲染结果
- 从左到右，Sample Num分别为16，256，1024



待实现： Path Tracing 绝缘体材质

- 框架内还支持实现其他暂时没有实现的材质类型，例如绝缘体材质和导体材质，请同学们自行探索
- 绝缘体，例如玻璃



待实现： Path Tracing 导体材质

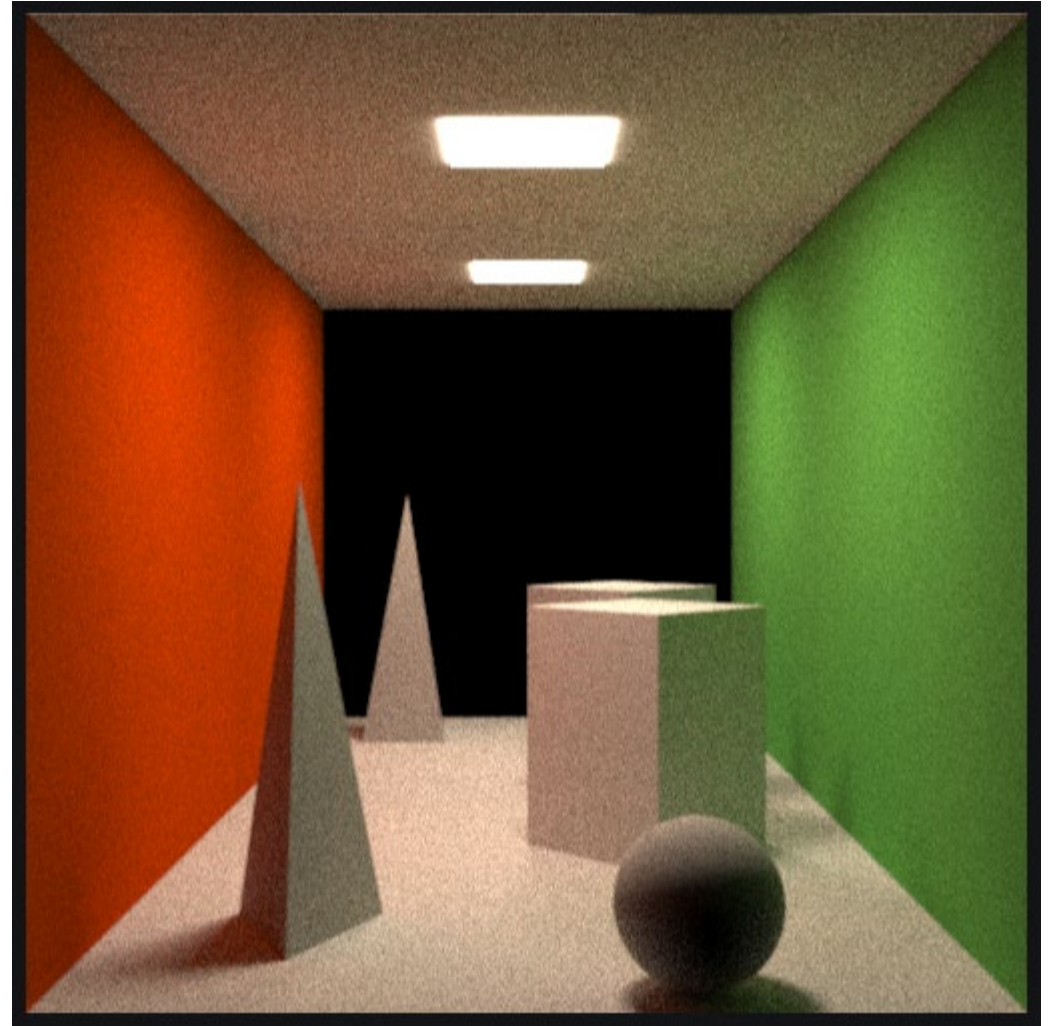
- 导体材质，即金属，具有镜面反射效果。

$$f_{BRDF}(x, \omega_i, \omega_o) = F_r(\omega_r) \frac{\delta(\omega_i - \omega_r)}{\cos \theta_i}$$

- 其中， δ 满足：
$$\delta(x) = \begin{cases} +\infty, & x = 0 \\ 0, & x \neq 0 \end{cases}$$

$$\int_{-\infty}^{+\infty} \delta(x) dx = 1$$

- ω_r 为反射光线
- $F_r(\omega_r)$ 为菲涅尔项



实现自定义渲染算法

- 打开./code/components文件夹
- 复制example文件夹至code/components文件夹中，重命名example文件夹为自定义的名称（例如：“MyTest”）
- 修改自定义文件夹中的CMakeLists.txt

```
...  
# 将"RenderExample"修改为你自己喜欢的名字, 千万不要和别的文件夹里的名字重复  
set(MY_COMPONENT_NAME "MyTest")  
...
```

实现自定义渲染算法

- 修改code/components/CMakeLists.txt

```
...  
# 添加  
add_subdirectory("./MyTest")  
.....
```

- 继承类NRenderer::RenderComponent, 并且重写void render(...)函数

```
// 详情请看RenderExample示例  
class Adapter : public NRenderer::RenderComponent {  
    virtual void render(NRenderer::SharedScene spScene) {  
        ....  
    }  
};
```

实现自定义渲染算法

- 注册插件

- + 参数1 -> 插件名称, 不要写中文
 - + 参数2 -> 插件的描述
 - + 参数3 -> 注册的类, 类的名字

```
REGISTER_RENDERER(Render, "MyTest", A);
```

- 重新生成CMake缓存并且重新编译插件

OBJ 文件格式入门

什么是 OBJ 文件

- OBJ 是一种常见的 3D 模型文件格式，由 Wavefront 提出。
- 它是 文本文件，可以用记事本打开和编辑。
- 用于描述：
 - 顶点坐标（模型几何形状）
 - 顶点法线（光照方向）
 - 纹理坐标（贴图映射）
 - 面（多边形） 的构成关系

基本语法结构

- OBJ 文件由行组成，每行开头用不同的关键字：
 - `v x y z` → 定义一个顶点 (vertex)
 - 例: `v 0.0 0.0 0.0`
 - `vn x y z` → 定义一个顶点法线 (vertex normal)
 - 例: `vn 0.0 1.0 0.0`
 - `vt u v` → 定义一个纹理坐标 (vertex texture)
 - 例: `vt 0.5 1.0`
 - `f ...` → 定义一个面 (face)，由顶点索引组成
 - 例: `f 1 2 3` 表示用第 1、2、3 个顶点组成一个三角形
 - 例: `f 1/1/1 2/2/1 3/3/1` 表示同时引用顶点/纹理/法线

简单样例-三角形

- 新建一个triangle.obj:

```
# 一个简单的三角形
```

```
v 0.0 0.0 0.0
```

```
v 1.0 0.0 0.0
```

```
v 0.0 1.0 0.0
```

```
f 1 2 3
```

简单样例-三角形

- 如何浏览obj文件：
 - 使用在线OBJ查看器
 - 自带工具：
 - Windows 10/11 自带3D Viewer应用，双击.obj打开
 - Mac课安装Xcode附带工具可以查看
 - Python+Open3D
 - 使用pip安装

```
import open3d as o3d

mesh = o3d.io.read_triangle_mesh("triangle.obj")
mesh.compute\_vertex\_normals\(\)
o3d.visualization.draw_geometries([mesh])
```

简单样例-长方体

- 长方体：
 - 顶点v: 8个角点
 - 面f: 每个面由2个三角形组成, 共12个三角形
 - 保存为cube.obj

```
# 一个长方体 (边长=2, 中心在原点)
# ---- 顶点 ----
v -1 -1 -1
v 1 -1 -1
v 1 1 -1
v -1 1 -1
v -1 -1 1
v 1 -1 1
v 1 1 1
v -1 1 1
```

简单样例-长方体

```
# ---- 面 ----  
# 每个面用两个三角形表示
```

```
# 底面 (-Z)  
f 1 2 3  
f 1 3 4
```

```
# 顶面 (+Z)  
f 5 6 7  
f 5 7 8
```

```
# 前面 (-Y)  
f 1 5 6  
f 1 6 2
```

```
# 后面 (+Y)  
f 4 3 7  
f 4 7 8
```

```
# 左面 (-X)  
f 1 4 8  
f 1 8 5
```

```
# 右面 (+X)  
f 2 6 7  
f 2 7 3
```